

GreenCharge

Gestão Inteligente de Eletropostos com Priorização de Energia Solar
Sprint 03 — Prototipagem Funcional e Integração

EV Challenge 2026 — Energias Renováveis e Sustentabilidade — FIAP
Semestre 2026.2

Bryan Lugli — RM 571350 — Turma 1CCPY
Beckman Lugli — RM 573442 — Turma 1CCPY
Guilherme Xavier — RM 573053 — Turma 1CCPY

1. O projeto

A ideia do GreenCharge surgiu de um problema bem concreto: a maioria dos eletropostos hoje puxa energia da rede mesmo quando tem painel solar gerando energia ao lado, porque simplesmente não existe um software conversando com as duas pontas. A gente resolveu atacar exatamente isso — um sistema que lê quanto de energia solar tá disponível, vê quantos carros estão conectados naquele momento, e decide quanto usar de sol e quanto puxar da rede, sempre dando prioridade pro sol primeiro.

Na Sprint 2 a gente já tinha validado essa lógica numa simulação em Python (o arquivo `greencharge_simulacao.py`), rodando 30 dias inteiros, hora a hora, usando o Clear Sky Model pra simular a geração solar (já que ainda não temos acesso à API real da GoodWe). Nessa Sprint 3 o objetivo era pegar essa mesma lógica e mostrar ela funcionando **de forma integrada e automatizada**, não só numa planilha de resultados — então montamos um protótipo simulado com um ESP32 no Wokwi, representando o controlador do eletroposto de verdade: ele recebe os dados de geração solar e carros conectados, aplica a mesma regra de decisão, e aciona comandos reais (os LEDs de status) de acordo com a fonte de energia escolhida.

2. Esquema de integração dos componentes

O diagrama de blocos abaixo mostra como os componentes se conectam, do dado de entrada até a ação automatizada no eletroposto:

ENTRADA -----	PROCESSAMENTO -----	SAIDA -----
API GoodWe (producao) -----> Modelo Clear Sky (POC) - 720 registros, 30 dias - ja validado na Sprint 2	Motor de Decisao (Python) -----> - le geracao solar kW - le veiculos conectados - decide kWh solar x kWh rede - calcula CO2 evitado e economia	Controlador EVSE (ESP32) - aplica a mesma regra de prioridade solar - aciona LED de status (verde/amarelo/vermelho) - imprime dados no Serial Dashboard / Relatorio (CSV, JSON, ODS 7/9/11/13)

Os blocos de "Entrada" e "Processamento" já estavam prontos e validados desde a Sprint 2 (as 720 horas simuladas). O que a gente adicionou nessa sprint foi o bloco de **"Saída"**: o protótipo no Wokwi (`greencharge_prototipo.ino` + `greencharge_diagram.json`) faz o papel do Controlador EVSE recebendo essa decisão e agindo em cima dela de verdade — acende o LED verde quando a recarga é 100% solar, amarelo quando é mista (solar + rede), e vermelho quando é 100% rede. É literalmente a mesma regra da função `decidir_fonte_recarga()` que já tínhamos em Python, só que reescrita em C++ pra rodar no microcontrolador.

3. Justificativa técnica das escolhas

Por que ESP32? É o microcontrolador que qualquer projeto de IoT conectado usa hoje em dia — já vem com Wi-Fi embutido, o que numa implementação real permitiria o controlador consultar a API SEMS da GoodWe direto e mandar comandos OCPP pro carregador, sem precisar de nenhum hardware extra. Simular ele no Wokwi deixa a gente demonstrar esse comportamento sem precisar de acesso físico a um eletroposto de verdade.

Por que 3 LEDs (verde/amarelo/vermelho)? Eles representam, de um jeito visual e imediato, a decisão que em produção viraria um comando OCPP mandado pro carregador (por exemplo, limitar a corrente ou redirecionar pra fonte solar). Os três estados cobrem exatamente os três cenários possíveis da nossa regra de decisão: 100% solar, misto, e 100% rede — os mesmos três casos que o

motor em Python já calcula e guarda no histórico de 30 dias.

Por que reaproveitar a lógica da Sprint 2 em vez de inventar uma nova? Porque a regra de decisão que colocamos no C++ do ESP32 é uma tradução direta da função `decidir_fonte_recarga()` que já tínhamos validado em Python. Isso garante que o protótipo não é uma peça solta — é a mesma inteligência de negócio da Sprint 2 rodando num dispositivo diferente, que é exatamente o tipo de integração que essa sprint pede pra gente mostrar.

4. Resultados e dados funcionais

O protótipo percorre 6 cenários representativos (extraídos dos padrões observados no histórico de 30 dias da Sprint 2), demonstrando os três estados de decisão:

Horário	Cenário	Solar (kW)	Veículos	Status	LED
07h	Manhã, sol fraco, pico de chegada	1,2	3	Recarga mista	Amarelo
12h	Meio-dia, sol forte, poucos carros	4,6	2	100% solar	Verde
14h	Tarde nublada, 4 carros	2,0	4	Recarga mista	Amarelo
19h	Fim de tarde, sol quase nulo, pico	0,3	4	100% rede	Vermelho
22h	Noite, sem veículos conectados	0,0	0	Em espera	Nenhum
03h	Madrugada, sem sol, 1 carro	0,0	1	100% rede	Vermelho

Esses cenários usam a mesma faixa de valores fisicamente possível da Sprint 2 (painel de 5 kWp, 4 pontos de 7,4 kW cada). Os resultados consolidados da simulação de 30 dias (já obtidos na Sprint 2, e que este protótipo passa a demonstrar em tempo real) foram:

Período simulado: 01/05/2026 a 30/05/2026 (720 horas)
Energia total consumida: 6.127,20 kWh
Energia solar utilizada: 561,84 kWh
Energia da rede elétrica: 5.565,36 kWh
Aproveitamento solar médio: 9,2 %
CO2 evitado no período: 50,40 kg
Projeção anual de CO2 evitado: 0,60 t
Economia na conta de luz: R\$ 477,56

O aproveitamento solar médio de 9,2% pode parecer baixo à primeira vista, mas faz sentido: com 4 carregadores de 7,4 kW, a demanda de pico chega a 29,6 kW, e o painel de 5 kWp entrega no máximo 5 kW — por isso nos horários de pico (07h, 19h) o protótipo mostra recarga mista ou 100% rede, e só no horário de menor demanda com sol forte (12h) a recarga fica 100% solar. O código aceita qualquer capacidade de painel (parâmetro `CAPACIDADE_PAINEL_KWP`), então com um painel maior esse número sobe bastante — já tínhamos estimado ~40% com 12 kWp lá na Sprint 1.

5. Conexão com os conteúdos da disciplina

Autoconsumo fotovoltaico: o sistema faz questão de usar primeiro a energia gerada no próprio local, só recorrendo à rede depois — isso reduz tanto o custo quanto a dependência de geração centralizada, que no Brasil ainda mistura fontes limpas com termelétricas.

Automação inteligente e demand response: o protótipo dessa sprint mostra, na prática, o princípio de ajustar a carga conforme a oferta de energia limpa disponível — quando tem sol, o controlador prioriza a recarga solar sozinho; quando não tem, ele aciona a rede sem ninguém precisar mexer em nada. É basicamente o conceito de resposta automatizada à disponibilidade energética que as smart grids usam.

Eficiência energética e rastreabilidade ambiental: cada decisão que o controlador toma gera dados que dá pra auditar (kWh solar, kWh rede, CO2 evitado, economia em reais), usando o Fator de

Emissão do SIN publicado pelo MCTI — a mesma métrica que empresas usam em relatório ESG.

ODS que a gente atende: ODS 7 (Energia limpa e acessível — prioriza solar nos eletropostos), ODS 9 (Indústria e inovação — automação em cima de infraestrutura que já existe, sem depender de hardware proprietário), ODS 11 (Cidades sustentáveis — mobilidade elétrica mais eficiente) e ODS 13 (Ação climática — CO2 evitado de um jeito que dá pra medir).